

FULL SYSTEM ARCHITECTURE AUDIT

FOREX ENGINE PRO

SHADOW OS v2 — Generated from source code (code is the source of truth)

VERSION 1.0 · OVERALL MATURITY "H 72%

Audit date: 2026-07-12 · 15 sections · 19 tables · ~56 endpoints

1 PROJECT SUMMARY

FOREX ENGINE PRO is a **live OANDA forex trading bot** that runs on Railway. The process tree is: Railway starts `telemetry/server.js` (the orchestrator + telemetry HTTP server), which **spawns `index.js` as a child process** (the actual trading brain) and supervises it with an exponential-backoff restart loop. Both processes write to a single shared **event-sourced spine** (the events table) through `telemetry/db-adapter.js`, which auto-selects PostgreSQL (when `DATABASE_URL` is set, i.e. production) or SQLite (local/dev).

On top of the live core sits the **SHADOW OS v2 manager tier** (`telemetry/managers/`) — an event-sourced, domain-manager architecture that is being introduced sprint by sprint to eventually replace the monolithic design without ever interrupting trading.

Attribute	Current state (from code)
Product	Live multi-pair OANDA forex bot (M5 signal / M1 entry)
Architecture version	SHADOW OS v2 (bootstrap <code>meta.systemVersion = v40.1</code>)
Runtime topology	<code>server.js</code> (parent) supervises <code>index.js</code> (child); shared Postgres/SQLite event spine
Completed migration sprints	0, 1, 2, 3, 4, 4.1, 5
Pending migration sprints	6 (KnowledgeManager), 7 (Recovery + Validation)
Live trading	Fully operational (real OANDA REST orders)
Shadow intelligence (A/B/C/D)	Built + connected to live gate; passive by default (OBSERVE)
Research/measurement layer	Built (Sprint 5); disabled by default (<code>SHADOW_LAB_RESEARCH OFF</code>)
Architecture health	Healthy — additive, idempotent, fail-safe; a few forward-declared tables
Overall readiness	Live core production-ready; intelligence layer built but dormant

Overall system maturity: ~72% (full computation in §15).

2 COMPLETE MODULE INVENTORY

The repository contains two disjoint worlds. **Only the first is the product.**

1. FOREX ENGINE PRO

— `index.js` + `telemetry/**`. The subject of this audit.

2. Replit monorepo scaffolding

— `artifacts/` (`api-server`, `mockup-sandbox`), `scripts/`, `lib/`, `tsconfig*`, `pnpm-workspace.yaml`. This is Replit's development harness; it is **not** part of the trading bot, is not started by Railway, and does not touch OANDA or the event spine. It is listed once here for completeness and then excluded from the rest of the audit.

Legend — Frozen = must never change; Prod = active in the running production process.

2.1 Status overview

Module	Type	Implemented	Enabled by default	Used in Prod	Frozen
index.js	Live trading brain	'	'	'	' FROZEN
telemetry/server.js	Orchestrator + HTTP API	'	'	'	No (additive-only)
telemetry/index.js	Telemetry core (logEvent, emitter)	'	'	'	No
telemetry/db-adapter.js	PG/SQLite abstraction	'	'	'	No
telemetry/shadowlab.js	Shadow engines A/B/C/D + shadowGate	'	' (OBSERVE)	'	A & B FROZEN
telemetry/shadowm.js	Shadow M exit tracker	'	'	' (passive)	No
telemetry/public/index.html	Dashboard (React via Babel)	'	'	'	No
managers/RuntimeDomainManager	Domain state (RDM)	'	' (via LMI)	'	No
managers/TradeIntentManager	Intent lifecycle (TIM)	'	' (via LMI)	'	No
managers/MemoryManager	Memory layer (MM)	'	' (via LMI)	'	No
managers/LiveMemoryIntegration	Live wiring (LMI)	'	' (SHADOW_OS_MEMORY)	'	No
managers/ShadowLabManager	Research reconciler	'	'L (SHADOW_LAB_RESEARCH off)	Wired, dormant	No
managers/shadowLabProvenance	Provenance helpers	'	' (used by SLM)	With SLM	No
migrations/autoMigrate.js	Startup migrator (ensureSchema)	'	' (Postgres only)	'	No
migrations/run.js	Manual psql -f runner	'	Manual	Dev/ops	No
archive/**	Dead code backups	n/a	'L	'L	Preserve, never run

2.2 Module detail

index.js — Live trading brain (FROZEN, 2360 L)

- **Purpose:** The real trader. Polls OANDA, runs the entry/exit/risk pipelines, places and manages orders.
- **Status:** Production, FROZEN. Carries no manager imports.
- **Who calls it:** Spawned as a child process by telemetry/server.js startBot().
- **What it calls:** OANDA REST (api-fxpractice/api-fxtrade) via axios; logEvent() from ./telemetry; shadowGate() from ./telemetry/shadowlab before every entry.
- **Dependencies:** axios, ./telemetry (logEvent), ./telemetry/shadowlab (shadowGate).

telemetry/server.js — Orchestrator + HTTP API (3135 L)

- **Purpose:** Supervises the live bot, parses its stdout into a live dashboard state, serves ~56 HTTP endpoints, and hosts the manager tier.
- **Status:** Production. Additive, flag-gated hooks only.
- **Who calls it:** Railway startCommand / npm start.
- **What it calls:** child_process.spawn("node", ["index.js"]); shadowLab.start(); shadowM.start(); new LiveMemoryIntegration(); new ShadowLabManager(); ensureSchema().

- **Dependencies:** express, ./index (logEvent/db/emitter), ./shadowlab, ./shadowm, ./managers, ./migrations/autoMigrate.

telemetry/index.js — Telemetry core (145 L)

- **Purpose:** logEvent() (fire-and-forget insert into events), in-process EventEmitter for SSE fan-out, DB stats/backup helpers.
- **Producer of:** every row in events.
- **Dependency quirk:** its _initSchema defines events.data as
TEXT
 ; migration 001 defines it as
JSONB
 (dual definer — whichever runs first wins; on a fresh boot _initSchema wins).

telemetry/db-adapter.js — DB abstraction (125 L)

- **Purpose:** One async interface (exec/all/get/run) over PostgreSQL (pg.Pool) or SQLite (node:sqlite).
- **Quirks (verified):** db.run() on an INSERT with no RETURNING auto-appends RETURNING id (breaks non-id PKs — use db.exec). db.exec() splits DDL on ; (unsafe for DO \$\$ blocks — those go through autoMigrate, not exec).

telemetry/shadowlab.js — Shadow engines + gate (1094 L)

- **Purpose:** Defines the four shadow engines and both a
live gate
 (shadowGate) and a
background reconciler
 (shadowLab.start()). See §4.
- **Who calls it:** index.js (shadowGate, live); server.js (shadowLab.start(), reconciler, and all /api/lab/* reads).

telemetry/shadowm.js — Shadow M exit tracker (718 L)

- **Purpose:** Observation-only evaluation of alternative exit strategies. See §4 (Shadow M).
- **Who calls it:** server.js (shadowM.start() + /api/shadowm/*).

telemetry/managers/* — SHADOW OS v2 tier

- **RuntimeDomainManager (RDM):** CAS-versioned domain state in runtime_domains (+ history + snapshots + consistency checks).
- **TradeIntentManager (TIM):** intent state machine in trade_intents (+ history). Actively used (12 code refs).
- **MemoryManager (MM):** append-first memory in memory_events (+ history) and a K/V cache in memory_entries.
- **LiveMemoryIntegration (LMI):** the only manager imported directly by server.js. Internally instantiates RDM + TIM + MM (this.rdm/tim/mm), runs startup recovery, per-trade hooks and graceful shutdown. Gated by SHADOW_OS_MEMORY (default on).
- **ShadowLabManager (SLM):** research reconciler !' shadow_* tables. Instantiated in server.js but only .start()ed when SHADOW_LAB_RESEARCH is on (default off).
- **shadowLabProvenance:** configHash/confidenceLevel/createProvenance helpers used by SLM.

3 LIVE BOT

All of the following is implemented in index.js (FROZEN).

3.1 Execution pipeline

- runBot() is an infinite while(true) loop.

- **Market hours:** when the FX market is closed (weekend, ~Sat 00:00 !' Sun 21:00 UTC) it sleeps and re-checks every 10 minutes.
- **Per cycle:** manageTrades() runs, then for each symbol in SYMBOLS it runs strategy(symbol) with a 2s pause between pairs and additional manageTrades() calls interleaved; then a 5s delay closes the cycle.
- **Timeframes:** MAIN_TIMEFRAME = M5 (signal), ENTRY_TIMEFRAME = M1 (entry). Pairs come from process.env.SYMBOLS.

3.2 Entry pipeline (strategy())

Signal !' sequential gates (block emits a signal_filtered event with a typed reason):

1. **Cooldown**
— 5-minute per-symbol lockout after a close (cooldown_block).
2. **Open-trade**
— one trade per symbol (open_trade_block).
3. **Correlation**
— a correlated pair being open blocks the new one (correlation_block).
4. **Disabled symbol**
— DISABLED_SYMBOLS env (symbol_disabled_block).
5. **Spread**
— blocks if spread > 2.0 pips (spread_block).
6. **Exhaustion**
— two-tier: candle strength (body/ATR) and price stretch vs EMA20, thresholds keyed to trendBucket (exhaustion_block).
7. **Minimum edge**
— $ATR \times 0.3 / \text{spread}$ must exceed ~1.15 (spread_edge_block).
8. **Pullback**
— price must be within ~1.5 pips of EMA20 (pullback_block).
9. **Margin**
— projected risk must fit available margin (margin_block).
10. **Gate v3 (9 conditions):**
M5 trend/candle/close/EMA/strength + M1 trend/candle/prev/close.
HARD
gate = all pass;
RELAXED
gate = score "e 6/9
and
anchor conditions (EMA, strength, candle) true.
11. **Shadow gate**
— await shadowGate(...); only blocks in GATE mode (\$4).

3.3 Exit pipeline (manageTrades())

- **SL/TP** fixed at entry from ATR ("H SL = $ATR \times 1.5$, TP = $SL \times 2$).
- **Break-even** — at +3.0 pips move SL to entry +0.5 (break_even).
- **Trailing** — at +10 pips trail by 6 pips.
- **MFE floor (v39.4)** — once MFE "e 2 pips, floor = $\max(0, MFE \times 0.35)$; SL is ratcheted on OANDA and a software exit fires if price gaps below floor (mfe_floor_set).
- **Profit protection** — exit if peak "e 4 and pips < peak " 1.5.
- **Momentum exit** — exit if peak "e 8 and pips < peak " 3.0.

- **Early exit** — exit if pips "d " 4.
- **Time exit** — exit if open "e 10 min and pips < 2.

3.4 Risk pipeline

- **Position sizing:** $\text{units} = (\text{Balance} * \text{RISK_PERCENT}) / (\text{SL_pips} * \text{pipMultiplier})$.
- **Margin check:** GET /v3/accounts/{id}/summary (NAV vs marginUsed) before entry.
- **Correlation guard:** correlated-pair exposure blocks new entries.
- **Defense mode:** after 3 consecutive losses, the EMA-distance gate tightens (~1.8 !' 2.5 pips); emits defense_mode_activated / defense_mode_cleared / defense_mode_skip.

3.5 Trade & order lifecycle

- **Broker:** OANDA REST via axios, Bearer OANDA_API_KEY, base api-fxpractice or api-fxtrade.
- **Open:** placeTrade() !' POST /v3/accounts/{id}/orders, MARKET order with stopLossOnFill + takeProfitOnFill; emits trade_open.
- **Detect fills/closes:** getOpenTrades() !' GET /v3/accounts/{id}/trades; a trade ID leaving the list means it closed; emits trade_close plus trade_forensics and periodic trade_state_snapshot.

3.6 State lifecycle

- **In index.js:** purely in-memory maps (cooldownMap, tradePeak, tradeBreakEven, tradeMAE, tradeFloorLevel, ...). The trading process keeps **no local persistence**; on restart it re-derives open positions from OANDA. This is by design and is why index.js can stay frozen and stateless.
- **In the telemetry layer:** the durable state lives in Postgres (event spine + manager tables) and is restored by the memory system below.

3.7 Memory lifecycle (parent process)

Handled in server.js via LiveMemoryIntegration (default on):

- **Startup:** restoreLiveState() !' memoryIntegration.init() !' recoverOnStartup({ liveState: live }) rebuilds the dashboard's live view so restarts show no ghost positions.
- **Runtime:** stdout parsing calls recordTradeOpen() / recordTradeClose(); a periodic timer (SHADOW_OS_PERSIST_MS) persists state.
- **Shutdown:** gracefulShutdown() ("H4 s budget) flushes on SIGTERM/SIGINT.
- **Fail-safe:** every hook is best-effort try/catch — a memory failure can never block trading.

4 SHADOW ENGINES

The shadow engines live in telemetry/shadowlab.js (A/B/C/D) and telemetry/shadowm.js (M).

There are **two evaluation paths** for A–D:

- **Live gate path:** index.js !' shadowGate(signal) !' A/B/C/D .evaluate(). Runs synchronously before each entry, always logs shadow_gate_eval, and logs shadow_gate_block only when it blocks.
- **Reconciler path:** server.js !' shadowLab.start() !' background cycle re-scores historical trade_opens and writes lab_shadow_a/b/c/d + lab_comparison (this is what feeds the dashboard /api/lab/*).

Mode: `_shadowMode = process.env.SHADOW_MODE || "OBSERVE"` (only GATE or OBSERVE; anything else forced to OBSERVE). Persisted via `shadow_mode_change` events; toggled at `POST /api/shadow/mode`.

- **OBSERVE (default):** engines never block — pure data collection.
- **GATE:** blocks a trade
 only
 when Engine D `wouldTrade === false`
 and
 confidence `=== "HIGH"`.
- **FAILSAFE:** any engine exception != blocked:false (trading continues).

Shadow A — ShadowQualityEngine (FROZEN)

- Exists ' · Implemented ' · Connected ' · Running ' · Passive ' (OBSERVE) · Influences live: only via Engine D vote in GATE · Collects data ' (lab_shadow_a) · Produces decision ' · Dashboard ' (Shadow A tab).
- **Computes:** weighted quality score 0–100 from the 9 condition flags + market bonuses; `wouldTrade` if score "e 65.
- **Maturity "H 85%"** Missing: score thresholds are static (no learning). Roadmap: feed Sprint 5 expectancy back into thresholds (research only).

Shadow B — ShadowContextEngine (FROZEN)

- Exists ' · Implemented ' · Connected ' · Running ' · Passive ' · Collects data ' (lab_shadow_b) · Produces decision ' · Dashboard '.
- **Computes:** market-state classification (TRENDING / RANGING / VOLATILE / NOISE / DEAD); `wouldTrade` only when TRENDING.
- **Maturity "H 85%"** Missing: state boundaries are heuristic constants. Roadmap: calibrate boundaries from measured expectancy per state.

Shadow C — ShadowKNNEngine (REBUILT)

- Exists ' · Implemented ' · Connected ' · Running ' · Passive ' · Collects data ' (lab_shadow_c) · Produces decision ' (tri-state) · Dashboard '.
- **Computes:** K-nearest-neighbours over a 15-dimensional feature vector against historical `trade_open+trade_close` pairs; `wouldTrade = true` if neighbour win-rate "e 55%, false if "d 45%; in the 45–55% zone it abstains (null)
 unless
 confidence is HIGH — HIGH with positive expectancy != true, HIGH with negative expectancy != false; needs "e 3 neighbours.
- **Maturity "H 70%"** Missing: cold-start needs volume; no incremental index (recomputes from events). Roadmap: persist the KNN dataset as a knowledge artifact (Sprint 6).

Shadow D — ShadowMetaEngine (NEW, v40)

- Exists ' · Implemented ' · Connected ' · Running ' · Passive ' (OBSERVE) · Influences live in GATE mode (it is the deciding vote) · Collects data ' (lab_shadow_d) · Produces decision ' · Dashboard '.
- **Computes:** dynamic weighted vote over A/B/C, weights learned from each engine's historical accuracy per symbol/session (`_refreshWeightsAsync`); `wouldTrade` if `voteScore` "e 0.55, false if "d 0.45, else null.
- **Maturity "H 70%"** Missing: GATE mode is not the production default (never live-blocking today); weight learning unproven at scale. Roadmap: shadow-validate against Sprint 5 expectancy, then a controlled GATE trial.

Shadow M — shadowm.js (Exit Lab)

- Exists ' · Implemented ' · Connected ' · Running **Strictly passive** (never opens/closes/modifies) · Collects data ' (shadowm_trades + shadowm_timeline) · Produces decision ' (best virtual exit) · Dashboard ' (Exit Lab).

- **Computes:** polls events every 5 s, consumes trade_open / trade_state_snapshot (~30 s) / trade_close, tracks MFE/MAE, simulates alternative exits (ATR-trail, profit-protect, time 1h/2h/4h, break-even, TP-extend) and on close ranks them vs realized profit !' best_strategy / profit_saved.
- **Maturity "H 80%":** missing: the "best strategy" is measured but never promoted into index.js exits (by design, frozen). Roadmap: expose recommendation through the research layer.

5 SHADOW LAB

"Shadow LAB" spans two implemented pieces:

- **Engine layer** (shadowlab.js) — the A/B/C/D evaluations described in §4.
- **Research/measurement layer** (managers/ShadowLabManager.js + shadow_* tables, Sprint 5) — turns the append-only event stream into a queryable, reproducible measurement layer.

Current capabilities

- Cursor-based, idempotent reconciler: trade_open !' shadow_signals, lab_shadow_a/b/c/d !' shadow_engine_evals, trade_close !' shadow_outcomes.
- Expectancy computation !' shadow_expectancy_snapshots (expectancy pips, profit factor, win/loss, confidence tier).
- **Provenance on every row:** run_id, build_id, config_hash (deterministic SHA-256 over canonical config) + dedupe_key idempotency, so every measurement is reproducible from the exact code+config that produced it.
- Confidence tiers auto-derived from sample size: LOW < 30, MEDIUM 30–100, HIGH > 100.

Current limitations

- **Disabled by default** — the reconciler only runs when SHADOW_LAB_RESEARCH=on; otherwise the shadow_* tables stay empty (endpoints still respond with researchEnabled:false).
- **Read/measure only** — by binding constraint it never feeds back into live trading.
- No automated ranking that promotes an engine to production; ranking today is descriptive (dashboard /api/lab/engine-ranking, computed from lab_shadow_* events, not from the research tables).

Current database tables

shadow_signals, shadow_engine_evals, shadow_outcomes, shadow_expectancy_snapshots (all migration 005; see §6).

Current APIs

- /api/lab/expectancy — expectancy snapshot(s) from the research layer.
- /api/lab/research/summary — reconciled counts/summary.
- /api/lab/research/timeseries — persisted expectancy time series.
- (Always registered; each reports researchEnabled.)

Current statistics & ranking

- **Research layer:** expectancy, profit factor, win/loss, sample & resolved counts, confidence tier — per scope (ALL / symbol / engine).
- **Engine layer (dashboard):** per-engine virtual win-rate, agreement/disagreement %, caution flags, and a heuristic engine ranking derived live from lab_shadow_* + lab_comparison.

Current research pipeline

events (append-only) !ShadowLabManager reconciler
(cursor + provenance + dedupe) !' shadow_signals / shadow_engine_evals / shadow_outcomes !' expectancy
aggregation !' shadow_expectancy_snapshots !' read-only research endpoints. Fully additive; safe to re-run (idempotent).

6 DATABASE

Backend is chosen at runtime by db-adapter.js (Postgres when DATABASE_URL set, else SQLite). In production, autoMigrate.ensureSchema() applies migrations 001–005 idempotently at startup and records them in schema_migrations. **19 tables** exist.

Retention is effectively permanent everywhere ("sacred constraint": no step may destroy accumulated knowledge); tables marked *append-only* forbid row deletion by design.

Table	Purpose	PK	Key relationships	Producer	Consumer	Migration
events	Append-only event spine	id	referenced by event_idempotency	logEvent (index.js, server.js, shadowlab.js)	all endpoints, reconcilers, ShadowM/LAB	001 (JSONB) / telemetry/index.js_initSchema (TEXT)
shadowm_trades	Shadow M per-trade exit analysis	id (signal_id unique)	keyed by signal_id to events	shadowm.js	/api/shadowm/*, dashboard	001 (narrow) / shadowm.js_initTables (wide)
shadowm_timeline	Shadow M pip/MFE/MAE timeline	id	signal_id !' shadowm_trades	shadowm.js	/api/shadowm/*	001 / shadowm.js
runtime_domains	CAS domain state (10 domains)	domain	history in runtime_domain_history	RuntimeDomainManager	RDM, LMI recovery	001 (+ bootstrap rows)
runtime_domain_history	Append-only RDM audit	id	snapshot_id !' system_snapshots	RuntimeDomainManager	RDM, audits	002
trade_intents	Intent state machine (OPEN/CLOSE/MODIFY)	id	history in trade_intent_history	TradeIntentManager	TIM	001 (+003 adds runtime_domain)
trade_intent_history	Append-only TIM audit	id	intent_id !' trade_intents	TradeIntentManager	TIM, audits	003
memory_entries	Memory K/V cache (namespaced, TTL)	id (namespace, key unique)	—	MemoryManager	MM	001
knowledge_artifacts	Versioned knowledge store	id	self-ref migration_from	none yet (Sprint 6)	none yet	001
event_idempotency	Event dedup keys	key	event_id !' events	none yet	none yet	001
consistency_log	Consistency-check findings	id	domain	RDM + MemoryManager	audits	001
system_snapshots	Full-system snapshots	id	referenced by RDM history	RuntimeDomainManager, /api/system/backup	RDM restore	001
memory_events	Append-first memory layer	id	history in memory_event_history	MemoryManager	MM (queries)	004
memory_event_history	Append-only memory audit	id	memory_id !' memory_events	MemoryManager	MM, audits	004

shadow_signals	1 row per observed signal (research)	id	signal_id, dedupe_key unique	ShadowLabManager	/api/lab/research/*	005
shadow_engine_evals	1 row per (signal, engine) eval	id	signal_id, dedupe_key unique	ShadowLabManager	research endpoints	005
shadow_outcomes	1 row per resolved signal (P/L)	id	signal_id, dedupe_key unique	ShadowLabManager	research endpoints	005
shadow_expectancy_snapshots	Expectancy time series	id	dedupe_key unique	ShadowLabManager	/api/lab/expectancy, timeseries	005
schema_migrations	Applied-migration ledger	filename	—	autoMigrate	autoMigrate	autoMigrate.js

- Notes (code truth):**
- events and the two shadowm_* tables have **two definers** each (a migration and a JS module). CREATE TABLE IF NOT EXISTS means the first to run wins; in production shadowm.js's **wide** shadowm_trades differs from migration 001's **narrow** one — a real schema-drift risk (§13).
 - knowledge_artifacts and event_idempotency are **forward-declared** (0 producers/consumers in code) — reserved for Sprint 6/future work.

7 API INVENTORY

All HTTP endpoints are served by telemetry/server.js (Express, PORT env, default 3001).

index.js and telemetry/index.js expose **no** HTTP routes. Grouped below; all are GET unless noted. "Consumer" = the dashboard tab or client that reads it.

7.1 Live & analytics

Method	Route	Purpose	Status	Consumer
GET	/	Serve dashboard index.html	Active	Browser
GET	/api/today	Today's live state / counts	Active	LIVE
GET	/api/events	Recent raw events	Active	DECYZJE
GET	/api/events/stream	SSE live event stream	Active	LIVE
GET	/api/trades	Trade history	Active	WYKRESY
GET	/api/stats	Aggregate performance	Active	ANALIZA
GET	/api/symbols	Per-symbol performance	Active	ANALIZA
GET	/api/live	Live open-position view	Active	LIVE
GET	/api/export	Export events	Active	EXPORT
GET	/api/winrate-analysis	Win-rate breakdown	Active	WYKRESY

7.2 Diagnostics & research analytics (events-derived)

Method	Route	Purpose	Status	Consumer
--------	-------	---------	--------	----------

GET	/api/exit-manager	Exit-logic diagnostics	Active	ANALIZA
GET	/api/spread-edge-analysis	Spread/edge study	Active	INSIGHTS
GET	/api/cooldown-analysis	Cooldown study	Active	INSIGHTS
GET	/api/fingerprints	Signal fingerprints	Active	WZORCE
GET	/api/excursion	MFE/MAE excursion	Active	EXCURSION
GET	/api/confirmation-lag	Confirmation lag	Active	ANALIZA
GET	/api/regime	Market regime	Active	INSIGHTS
GET	/api/session-performance	Session breakdown	Active	INSIGHTS
GET	/api/blocked-outcomes	Outcome of blocked signals	Active	INSIGHTS
GET	/api/drift	Strategy drift alerts	Active	INSIGHTS
GET	/api/insights	Consolidated insights	Active	INSIGHTS
GET	/api/m1trend-experiment	M1 trend experiment	Active	INSIGHTS
GET	/api/m5trend-experiment	M5 trend experiment	Active	INSIGHTS
GET	/api/m1close-experiment	M1 close experiment	Active	INSIGHTS
GET	/api/gate-experiment	Gate experiment	Active	INSIGHTS
GET	/api/almost-trades	Near-miss signals	Active	BLOCKED
GET	/api/blocked-winners-v2	Blocked would-be winners	Active	BLOCKED
GET	/api/condition-performance	Per-condition performance	Active	BLOCKED
GET	/api/post-entry-failures	Post-entry failures	Active	INSIGHTS
GET	/api/trade-quality	Trade-quality metrics	Active	INSIGHTS
GET	/api/pipeline-audit	End-to-end pipeline audit	Active	PIPELINE
GET	/api/weak-relaxed	Weak/relaxed-gate trades	Active	FILTERED

7.3 Shadow engine LAB (from lab_shadow_* / lab_comparison events)

Method	Route	Purpose	Status	Consumer
GET	/api/lab/overview	Combined engine overview	Active	LAB
GET	/api/lab/shadow-a	Engine A decisions	Active	LAB
GET	/api/lab/shadow-b	Engine B decisions	Active	LAB
GET	/api/lab/shadow-c	Engine C decisions	Active	LAB
GET	/api/lab/shadow-d	Engine D decisions	Active	LAB
GET	/api/lab/comparison	Engines vs live agreement	Active	LAB
GET	/api/lab/virtual-performance	Virtual P/L per engine	Active	LAB
GET	/api/lab/engine-ranking	Heuristic engine ranking	Active	LAB
GET	/api/lab/unified-report	One-shot combined report	Active	tools

7.4 Shadow control, Shadow M, research layer & health

Method	Route	Purpose	Status	Consumer
GET	/api/shadow/status	Mode + shadow memory stats	Active	LAB

POST	/api/shadow/mode	Switch OBSERVE !" GATE	Active	ops
POST	/api/system/backup	Snapshot/backup trigger	Active	ops
GET	/api/shadowm/status	Shadow M status	Active	LAB
GET	/api/shadowm/trades	Shadow M trades	Active	LAB
GET	/api/shadowm/active	Shadow M active trackers	Active	LAB
GET	/api/shadowm/dashboard	Shadow M dashboard bundle	Active	LAB (Exit Lab)
GET	/api/shadowm/diag	Shadow M diagnostics	Active	ops
POST	/api/admin/shadowm/force-close	Inject synthetic close (ghost fix)	Active	ops/admin
GET	/api/healthz/persistence	Persistence health	Active	ops
GET	/api/memory-integration/status	LMI status	Active	ops
GET	/api/lab/expectancy	Research expectancy	Active (reports researchEnabled)	research
GET	/api/lab/research/summary	Research summary	Active (dormant when off)	research
GET	/api/lab/research/timeseries	Expectancy timeseries	Active (dormant when off)	research

8 DASHBOARD

Single-page app at `telemetry/public/index.html` (React via Babel-standalone, served at

/). A top bar shows bot status + UTC clock; a horizontal tab bar drives the views.

Tab	Key widgets	Endpoints feeding it	Tables behind them
LIVE	Status cards, open positions, active blocks, live log	/api/today, /api/events/stream (SSE)	events
ANALIZA	Performance cards, block breakdown, per-symbol table, confirmation lag, exit-manager	/api/stats, /api/symbols, /api/confirmation-lag, /api/exit-manager	events
PIPELINE	End-to-end funnel audit	/api/pipeline-audit	events
WYKRESY	Trade charts, win-rate analysis	/api/trades, /api/winrate-analysis	events
DECYZJE	Recent decisions feed	/api/events?limit=100	events
WZORCE	Signal fingerprints	/api/fingerprints	events
EXCURSION	MFE/MAE excursion	/api/excursion	events
INSIGHTS	Outcomes, drift, trend/close/gate experiments, post-entry failures, trade quality, spread-edge, cooldown	/api/insights, /api/blocked-outcomes, /api/drift, /api/m1trend-experiment, /api/m5trend-experiment, /api/m1close-experiment, /api/gate-experiment, /api/post-entry-failures, /api/trade-quality, /api/spread-edge-analysis, /api/cooldown-analysis	events
BLOCKED	Almost-trades, blocked winners, condition performance	/api/almost-trades, /api/blocked-winners-v2, /api/condition-performance	events
FILTERED	Weak/relaxed-gate trades	/api/weak-relaxed	events
LAB	Sub-tabs: Overview, Shadow A, B, C, D, Comparison, Virtual Perf, Ranking, Exit Lab	/api/lab/overview, /api/lab/shadow-a..., /api/lab/comparison, /api/lab/virtual-performance, /api/lab/engine-ranking, /api/shadow/status, /api/shadowm/dashboard	events (lab_shadow_*, lab_comparison), shadowm_trades/timeline

EXPO RT	Data export	/api/export	events
------------	-------------	-------------	--------

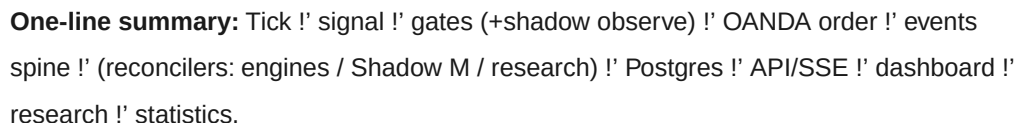
Observations (code truth): the dashboard reads almost entirely from the events spine (plus Shadow M tables via `/api/shadowm/dashboard`). It does **not** yet surface the Sprint 5 research endpoints (`/api/lab/expectancy`, `/api/lab/research/*`) — those exist server-side but have no dedicated widget. Labels are Polish (ANALIZA, WYKRESY, DECYZJE, WZORCE).

9 MEMORY SYSTEM

The memory system is the SHADOW OS v2 manager tier, wired into production through LiveMemoryIntegration (default on via SHADOW_OS_MEMORY).

- **Memory layer (MemoryManager):** append-first event memory in `memory_events` with a full-row audit trail in `memory_event_history`, plus a namespaced K/V cache with TTL in `memory_entries`. Tri-state coercion and dedupe-key idempotency are enforced in code.
- **Domain state (RuntimeDomainManager):** CAS-versioned `runtime_domains` (10 domains: `live`, `shadowA/B/C/D`, `shadowM`, `exitLab`, `telemetry`, `scheduler`, `meta`) with `runtime_domain_history` audit and `system_snapshots` for point-in-time capture.
- **Intent tracking (TradeIntentManager):** `trade_intents` state machine (PENDING !' CONFIRMED/FAILED/RECONCILED) with `trade_intent_history` audit.
- **Recovery:** on startup `LMI.recoverOnStartup()` rebuilds the live dashboard view and reconciles open intents so restarts show no ghost positions.
- **Snapshots:** `system_snapshots` (RDM) + POST `/api/system/backup`; on SQLite, `telemetry/index.js` also keeps rolling file backups.
- **Knowledge:** `knowledge_artifacts` table exists but has **no producer yet**
—
KnowledgeManager is Sprint 6 (not implemented).
- **Persistence / PostgreSQL:** production uses Postgres via `db-adapter` (`pg.Pool`); `autoMigrate` applies 001–005 idempotently at boot. Consistency findings are recorded in `consistency_log` by RDM and MM.
- **Redis:**
not present
— there is no Redis client, dependency, or connection anywhere in the code. All persistence is Postgres (prod) / SQLite (dev).

10 COMPLETE DATA FLOW



Implemented = code exists · Active = runs in the default production config · Shadow = passive/observational · DB = owns/uses tables · API = has endpoints · Research = feeds Sprint 5 research layer.

Module	Implemen ted	Active	Producti on	Shado w	Dashb oard	Database	API	Researc h	Status
Live bot (index.js)	'	'	'	—	'	'	—	source	Production
Orchestrator (server.js)	'	'	'	—	'	'	'	—	Production
Telemetry core	'	'	'	—	'	'	—	source	Production
DB adapter	'	'	'	—	—	'	—	—	Production
Shadow A	'	'	'	'	'	'	'	'	Passive (OBSERVE)
Shadow B	'	'	'	'	'	'	'	'	Passive (OBSERVE)
Shadow C	'	'	'	'	'	'	'	'	Passive (OBSERVE)
Shadow D	'	'	'	'	'	'	'	'	Passive (gate-capable)
Shadow M	'	'	'	'	'	'	'	via events	Passive
RuntimeDomainManager	'	'	'	—	—	'	via LMI	—	Production
TradeIntentManager	'	'	'	—	—	'	via LMI	—	Production
MemoryManager	'	'	'	—	—	'	via LMI	—	Production
LiveMemoryIntegration	'	'	'	—	status	'	'	—	Production
ShadowLabManager (research)	'	'L	wired	'	—	'	'	'	Disabled by default
KnowledgeManager	'L	'L	—	—	—	table only	—	—	Future (Sprint 6)
RecoveryManager	'L	'L	—	—	—	—	—	—	Future (Sprint 7)
ValidationManager	'L	'L	—	—	—	consistency_lo g	—	—	Future (Sprint 7)

12 PRODUCTION READINESS

- **Ready (live in default prod config):**
index.js, telemetry/server.js, telemetry/index.js, db-adapter.js, autoMigrate.js, RuntimeDomainManager, TradeIntentManager, MemoryManager, LiveMemoryIntegration, Shadow M, the dashboard, and the analytics/LAB endpoints.
- **Experimental (built, running, but not authoritative):**
Shadow A/B/C/D in OBSERVE mode — they evaluate and log but never block trades.
Engine D GATE mode is code-complete but not the default.
- **Prototype / built but dormant:**
ShadowLabManager research layer + shadow_* tables — fully implemented and wired but gated OFF (SHADOW_LAB_RESEARCH). Endpoints respond with researchEnabled:false.
- **Disabled / forward-declared:**
knowledge_anchors and event_idempotency tables (no producers).
GATE mode (off by default).
- **Future (not implemented):**
KnowledgeManager (Sprint 6), RecoveryManager + ValidationManager (Sprint 7).

13 TECHNICAL DEBT

- **Known limitations**
• **events.data type mismatch (dual definer):** migration 001 = JSONB, telemetry/index.js _initSchema = TEXT. On a fresh prod boot _initSchema wins !" events.data is TEXT. Harmless today (no JSONB operators used on it) but must be reconciled before any JSONB query is added.

- **shadowm_trades / shadowm_timeline schema drift (dual definer):** shadowm.js creates a **wide** shadowm_trades (sl/tp/atr/ex_* columns); migration 001 creates a **narrow** one. Whichever CREATE TABLE IF NOT EXISTS runs first wins — a latent inconsistency between environments.
- **Research layer invisible in UI:** Sprint 5 endpoints exist but no dashboard widget consumes them.
- **Shadow C/D need volume:** KNN and meta-weight learning are cold until enough resolved trades accumulate.

Architecture debt

- **Two schema-definition sources** (SQL migrations vs. JS CREATE TABLE) for events and the shadowm_* tables. Long term, migrations should be the single source; JS modules should stop issuing CREATE TABLE.
- **db.exec() ;-splitter** is unsafe for multi-statement DDL with DO \$\$ blocks; the safe path (autoMigrate whole-file pool.query) exists but the unsafe helper remains available.
- **db.run() auto-RETURNING id** silently breaks tables with non-id PKs (runtime_domains); callers must remember to use db.exec().
- **Forward-declared tables** (knowledge_artifacts, event_idempotency) sit unused, implying planned managers not yet built.
- **Monolith duality:** the frozen index.js still owns all live decisioning; the manager tier observes and remembers but does not yet drive trading.

Future refactors

- Reconcile the two events / shadowm_* definitions into migrations only.
- Promote Shadow M's measured best-exit and Sprint 5 expectancy into the dashboard (read-only) before any live influence.
- Build KnowledgeManager to give knowledge_artifacts a producer (persist KNN datasets, engine weights).

Risks

- **Concurrency:** prior gotchas note a CAS pool deadlock pattern and ANY(\$1)-array pitfalls — respect the documented client-holding and array-filter rules in the managers.
- **GATE mode:** enabling SHADOW_MODE=GATE lets Engine D block live trades; must be volume-validated first (fail-safe already defaults to allow-on-error).
- **Ephemeral storage:** without DATABASE_URL (or DATA_DIR) on Railway, SQLite is ephemeral — production must keep Postgres attached (the sacred constraint).

14 FUTURE ROADMAP

Recommended order (uses only architecture already present in the code — no invented features):

1. Reconcile schema definitions (debt paydown).

Make migrations the single source

for events and shadowm_*; remove JS CREATE TABLE drift. Low risk, unblocks JSONB use and cross-env consistency.

2. Turn on the research layer in shadow.

Enable SHADOW_LAB_RESEARCH and let

ShadowLabManager populate shadow_*; add read-only dashboard widgets for /api/lab/expectancy and /api/lab/research/*. Pure measurement, zero live impact.

3. Sprint 6 — KnowledgeManager.

Give knowledge_artifacts a producer: persist the

Shadow C KNN dataset and Shadow D weights as versioned, checksummed artifacts.

4. Sprint 7 — RecoveryManager + ValidationManager.

Formalize crash recovery and wire

consistency_log into an active validation pass (RDM/MM already write to it).

5. **Controlled GATE trial.**

Only after (2)–(4) show positive measured expectancy,
shadow-validate Engine D in GATE mode on a limited scope, keeping the fail-safe.

15 EXECUTIVE SUMMARY

What already exists. A complete, running live forex bot (index.js, frozen) under an orchestrating telemetry server (server.js); an event-sourced spine (events) on Postgres/SQLite; four shadow decision engines (A/B/C/D) plus a Shadow M exit tracker; a full SHADOW OS v2 memory tier (RDM + TIM + MM + LMI); a Sprint 5 research/measurement layer (ShadowLabManager + shadow_* tables); ~56 HTTP endpoints; and a Polish-language React dashboard.

What actually works (default prod config). Live trading end-to-end against OANDA; the full analytics/LAB dashboard off the event spine; the memory system (recovery, per-trade memory, snapshots, consistency logging); Shadow M exit measurement; and the shadow engines running in OBSERVE (they evaluate and log every signal).

What is connected. index.js !' shadowGate (live, synchronous) and !' logEvent; server.js !' child bot, reconcilers, memory hooks, and all APIs; LMI !' RDM/TIM/MM; ShadowLabManager !' wired into server.js. The dashboard is connected to the event spine and Shadow M.

What is disconnected / dormant. The Sprint 5 research reconciler (OFF by default) and its endpoints (respond but empty); knowledge_artifacts and event_idempotency (no producers); the research endpoints have no dashboard widgets; GATE mode (off).

What influences live trading. Only index.js's own pipelines. The shadow engines can influence live **only** if SHADOW_MODE=GATE (Engine D, HIGH-confidence SKIP) — not the default. Everything else is observe/measure.

What only collects data. Shadow A/B/C/D (OBSERVE), Shadow M, the memory tier's audit trails, the research layer (when enabled), and all analytics endpoints.

What remains to be implemented. KnowledgeManager (Sprint 6); RecoveryManager + ValidationManager (Sprint 7); dashboard surfacing of the research layer; schema-definition reconciliation; and a validated path to activate GATE mode.

Maturity score (0–100%)

Dimension	Weight	Score	Contribution
Live trading core (operational)	30%	95%	28.5
Telemetry + event spine + DB	15%	90%	13.5
Memory tier (RDM/TIM/MM/LMI, live)	15%	85%	12.75
Shadow engines A/B/C/D (built, passive)	15%	70%	10.5

Shadow M (built, passive)	5%	80%	4.0
Research layer (built, dormant)	10%	50%	5.0
Knowledge/Recovery/Validation (future)	10%	5%	0.5
TOTAL	100%		"H 72%

Current overall maturity of FOREX ENGINE PRO: "H 72%.
The live product is production-grade, the SHADOW OS v2 intelligence and knowledge layers are largely built (5 of 7 sprints) but deliberately dormant, awaiting measurement-driven activation.

End of FOREX ENGINE PRO — Full System Architecture Audit v1.0. Generated from source code on 2026-07-12.